

浙江大学

ZHEJIANG UNIVERSITY



电子电路设计实验 实验报告

实验名称 示例项目与 A3 项目

实验地点 东四 216

姓 名 金宣妤

学 号 3240100333

实验日期 2026.5.11

指导老师 叶险峰、施红军、邓靖靖

Contents

1 实验一：I-V 转换电路设计	1
1.1 实验概述	1
1.2 原理图分析	1
1.2.1 第一级：差动放大电路	1
1.2.2 第二级：反相加法器电路	1
1.3 PCB 设计的学习收获	2
1.3.1 封装选择与引脚对应	2
1.3.2 布局原则	2
1.3.3 布线规范	3
1.4 遇到的问题与调试方法	3
1.4.1 分析过程	3
1.4.2 独立调试方法	4
1.4.3 更换 R_7 解决增益不足	4
2 实验二：基于 Arduino 的温度检测显示控制器	5
2.1 实验概述	5
2.2 硬件方案	5
2.2.1 各模块说明	5
2.3 软件设计	6
2.3.1 温度控制逻辑——迟滞设计	6
2.3.2 显示刷新策略——防闪烁	6
2.3.3 旋转编码器读取	7
2.3.4 本组软件改进	7
2.4 原理图与 PCB 设计	10
2.4.1 PCB 设计要点	11
2.5 本实验学习收获	12
总结	12

1 实验一：I-V 转换电路设计

1.1 实验概述

本实验设计了一个电流-电压转换电路，将工业控制中常用的 $4 \sim 20 \text{ mA}$ 标准电流信号转换为 $0 \sim 10 \text{ V}$ 的标准电压信号。整个电路由两级运放电路构成，核心器件选用精密运放 OP-07。

转换关系如下：

$$V_o = \frac{R_f}{R_6} I_i R_1 - \frac{R_f}{R_7} V_f \quad (1)$$

1.2 原理图分析

1.2.1 第一级：差动放大电路

电流源 I_i ($4 \sim 20 \text{ mA}$) 流过采样电阻 $R_1 = 250 \Omega$ ，产生 $1 \sim 5 \text{ V}$ 的压降。运放 A_1 与 $R_2 = R_3 = R_4 = R_5 = 10 \text{ k}$ 构成单位增益差动放大器，输出为：

$$V_{o1} = -I_i R_1 \in [-5 \text{ V}, -1 \text{ V}] \quad (2)$$

与 R_1 并联的电容 $C_1 = 100 \text{ pF}$ 用于抑制高频干扰，构成简单的低通滤波。

1.2.2 第二级：反相加法器电路

A_2 与 R_6 、 R_7 、 R_f 构成反相加法器，将 V_{o1} 的 $-1 \sim -5 \text{ V}$ 变换至 V_o 的 $0 \sim 10 \text{ V}$ 。由式 (1)，增益需满足：

$$\frac{R_f}{R_6} = \frac{\Delta V_o}{\Delta V_{o1}} = \frac{10 \text{ V}}{4 \text{ V}} = 2.5 \quad (3)$$

取 $R_6 = 10 \text{ k}$ ，则 $R_f = 25 \text{ k}$ （由 20 k 固定电阻串联 $R_{w1} = 10 \text{ k}$ 电位器组成）。

偏置电压 V_f 由 5.1 V 稳压管基准后经 R_{12} 、 R_{w2} 分压产生，理论值约为 $+1 \text{ V}$ （使 $I_i = 4 \text{ mA}$ 时 $V_o = 0 \text{ V}$ ）。

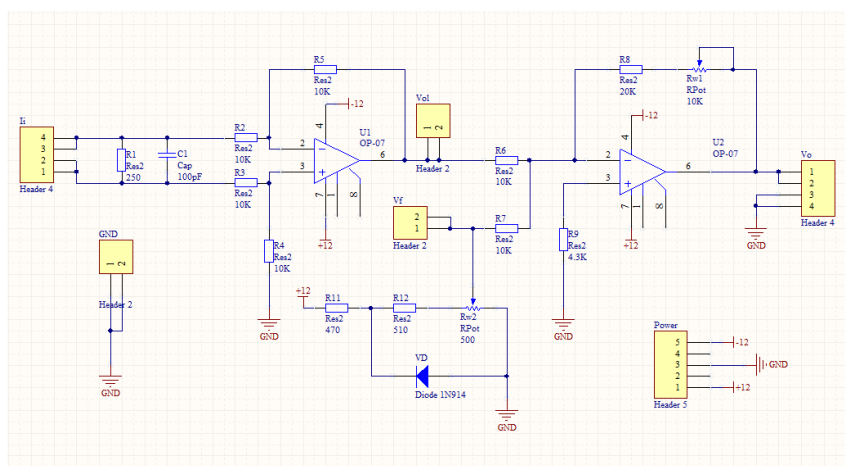


Figure 1: 示例项目原理图

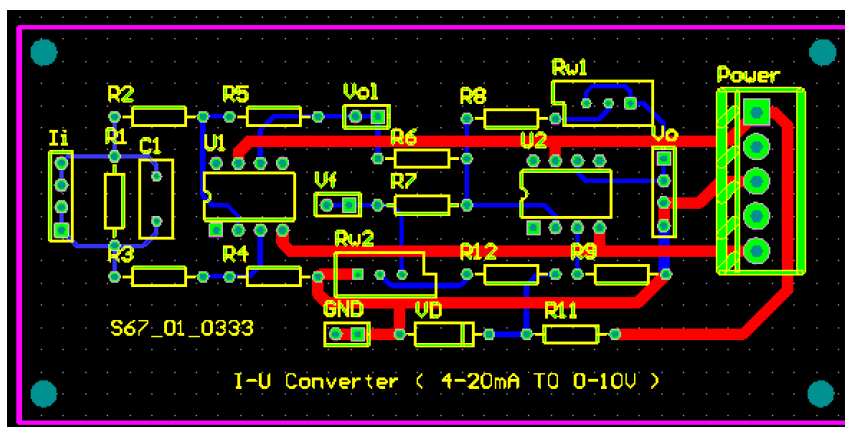


Figure 2: 示例项目 PCB

1.3 PCB 设计的学习收获

完成本实验的 PCB 设计是我第一次系统地将原理图转化为实际板图。

1.3.1 封装选择与引脚对应

原理图设计阶段就需要为每个元件指定封装，否则导入 PCB 时会有元件遗漏。本实验所有元件采用通孔直插封装。

此外，原理图符号中的引脚名与 PCB 封装引脚名可能不一致，在二极管中尤为常见——例如某些稳压管原理图引脚名为 1、2，而封装引脚名为 A、K，这会导致飞线缺失。解决方法是在原理图中手动编辑引脚名使其一致。

1.3.2 布局原则

布局决定了布线的难度，是 PCB 设计中最值得花时间的环节：

- (1). 同一子电路的元件尽量放在一起。本设计分为：差动放大模块 (A_1 、 $R_1—R_5$ 、 C_1)、反相加法器模块 (A_2 、 $R_6—R_9$ 、 R_{w1})、稳压分压模块 (V_D 、 R_{11} 、 R_{12} 、 R_{w2})。
- (2). 各模块按信号从左到右的顺序排列，与原理图走向一致，便于调试时对照查找。
- (3). 模块内部元件位置尽量与原理图保持一致，旋转而不镜像（集成块镜像后引脚顺序反转，是致命错误）。

1.3.3 布线规范

- 信号线宽取 20 mil，电源线和地线取 50 mil，有利于降低电源阻抗；
- 对角焊盘之间走线加 45° 折角，避免纯直角（90° 拐角在高频下会产生反射，虽然本设计频率不高，但养成习惯对以后有益）；
- 网格（Grid）步进值调整为 50 mil，保证元件排列整齐；
- 丝印层标注电路名称、作者信息及电源极性，便于焊接和使用。

1.4 遇到的问题与调试方法

调试初期，无论怎样旋转两个电位器 R_{w1} 、 R_{w2} ，输出电压始终无法同时满足： $I_i = 4\text{ mA}$ 时 $V_o = 0\text{ V}$ ，且 $I_i = 20\text{ mA}$ 时 $V_o = 10\text{ V}$ 。两个条件相互牵制，调好一个另一个就偏了。

1.4.1 分析过程

重新推导式 (1)，将两个端点代入：

$$I_i = 4\text{ mA} : V_o = \frac{R_f}{R_6} \cdot (-1\text{ V}) - \frac{R_f}{R_7} V_f = 0 \quad (4)$$

$$I_i = 20\text{ mA} : V_o = \frac{R_f}{R_6} \cdot (-5\text{ V}) - \frac{R_f}{R_7} V_f = 10\text{ V} \quad (5)$$

(5)–(4)，消去含 V_f 的项：

$$\frac{R_f}{R_6} \cdot (-5 + 1)\text{ V} = 10\text{ V} \Rightarrow \frac{R_f}{R_6} = 2.5 \quad (6)$$

也就是说，两个电位器分别控制独立的物理量：

- R_{w1} （与 R_8 串联构成 R_f ）：控制增益，即输出变化的幅度（斜率）
- R_{w2} （分压产生 V_f ）：控制零点偏移，即 4 mA 对应的输出电压

1.4.2 独立调试方法

基于上述分析，我制定了以下两步调试流程，使两个参数解耦、互不干扰：

- (1). **第一步——调零点（固定增益）**：令输入电流 $I_i = 4\text{ mA}$ ，此时无论增益取何值，式 (4) 中含 V_f 的项决定了输出是否为零。调节 R_{w2} （偏置电位器）直至 $V_o = 0\text{ V}$ 。
- (2). **第二步——调增益（零点已锁定）**：令输入电流 $I_i = 20\text{ mA}$ ，此时零点已固定，调节 R_{w1} （增益电位器）直至 $V_o = 10\text{ V}$ 。

1.4.3 更换 R_7 解决增益不足

按上述方法调试后发现，即使 R_{w1} 旋转到最大值， $I_i = 20\text{ mA}$ 时的输出仍低于 10 V ，说明增益不足。

由增益公式：

$$\frac{R_f}{R_6} = \frac{R_8 + R_{w1}}{R_6} = 2.5 \quad \Rightarrow \quad R_f = 2.5 \times R_6 = 25\text{ k} \quad (7)$$

检查发现实际焊接的 R_7 阻值偏大，导致 V_f 对输出的影响过强，压缩了 R_{w1} 的有效调节范围。将 R_7 换为阻值更小的电阻后，调节范围恢复正常，按两步法顺利完成调试。

本实验学习收获总结

- (1). 遇到调试困难时，不要盲目旋转旋钮，先回到原理图推导各参数的物理含义，才能找到正确的调试方向。
- (2). 将两个独立变量（零点和增益）分别对应两个可调元件，逐步调节，是模拟电路调试的通用方法。
- (3). 调试异常时，不要只改调节量，也要回头核实固定元件的实际值是否与设计一致。

2 实验二：基于 Arduino 的温度检测显示控制器

2.1 实验概述

本实验基于 Arduino UNO 扩展板，设计并实现了一套温度检测显示控制系统。系统以 LM35 为温度传感器，LCD1602 为显示屏，EC11 旋转编码器为交互控制器，通过继电器驱动功率电阻加热。核心功能要求如表 1 所示。

Table 1: 系统功能规格

功能	指标	说明
温度测量范围	0 ~ 100 °C	LM35 单电源模式
测量精度	±0.5 °C	LM35 固有精度
温度设定	旋转编码器调节	普通/快速两档
迟滞控制	±0.25 °C	防止继电器频繁通断
Override 模式	无加热上限	按下编码器按钮触发

2.2 硬件方案

2.2.1 各模块说明

温度传感器 LM35 LM35 采用单电源供电（0 ~ 100 °C 量程），输出电压与温度成线性关系：

$$U_{\text{out}} = 10 \text{ mV}/^{\circ}\text{C} \times t/^{\circ}\text{C}$$

(8)

Arduino 模拟口 A0 读取后，通过以下换算得到温度值：

$$t = \frac{V_{\text{ADC}}}{1023} \times 5000 \text{ mV} \div 10 \text{ mV}/^{\circ}\text{C}$$

(9)

EC11 旋转编码器 编码器具有 A、B 两路正交输出，通过判断 A、B 的跳变顺序确定旋转方向。同时，编码器按钮可触发快速调温（10 倍速）与 Override 模式切换。编码器输入端加 100 nF 电容消抖，有效解决了前期测试中的误触发问题。

LCD1602 显示模块 工作于 4 位总线模式，节省 Arduino I/O 口。显示内容分两行：

- 第一行：实测温度 (Actual: XX.XX °C)
- 第二行：设定温度或 Override 状态 (Set: XX.XX °C / OVERRIDE ON)

控制驱动电路 Arduino 数字口通过 1k 限流电阻驱动 NPN 三极管基极，三极管集电极控制继电器线圈（HK4100F，5V 版）。续流二极管 D2 防止继电器断开时产生的反向感应电动势损坏三极管。LED D1 指示加热状态。

2.3 软件设计

2.3.1 温度控制逻辑——迟滞设计

简单通断控制会导致继电器在设定温度附近频繁切换，损坏器件。本设计引入迟滞变量 $\delta = 0.25^{\circ}\text{C}$ ，实现带死区的控制：

Listing 1: 温度控制核心逻辑

```
1 float hysteresis = 0.25;
2
3 void updateOutputs() {
4     if (override || measureTemp < setTemp - hysteresis) {
5         // 温度低于下限，或处于 Override 模式：开始加热
6         digitalWrite(ledPin, HIGH);
7         digitalWrite(relayPin, HIGH);
8         ride = 1;
9     } else if (!override && measureTemp > setTemp + hysteresis) {
10        // 温度超过上限：停止加热
11        digitalWrite(ledPin, LOW);
12        digitalWrite(relayPin, LOW);
13        ride = 0;
14    }
15    // 介于两者之间：维持当前状态（死区）
16 }
```

2.3.2 显示刷新策略——防闪烁

若每次主循环都刷新 LCD，温度读数的微小波动会导致屏幕持续闪烁。解决方法是引入计数器，每 1000 次循环才刷新一次显示；并采用直接覆写而非 `lcd.clear()` 的方式更新数值，避免整屏重绘带来的闪烁。

Listing 2: 计数器控制刷新频率

```
1 if (count == 1000) {
2     updateDisplay(); // 覆写新值，不调用 lcd.clear()
3     updateOutputs();
```



```
4     count = 0;
5 }
6 count++;
```

2.3.3 旋转编码器读取

通过检测 A 路上升沿时 B 路的电平判断旋转方向，按下按钮时速度提升 10 倍，无需快速旋转即可大幅调整设定温度：

Listing 3: 编码器方向识别

```
1 int getEncoderTurn() {
2     static int oldA = LOW, oldB = LOW;
3     int result = 0;
4     int newA = digitalRead(aPin);
5     int newB = digitalRead(bPin);
6     if (newA != oldA || newB != oldB) {
7         if (oldA == LOW && newA == HIGH)
8             result = -(oldB * 2 - 1); // 判断方向
9     }
10    oldA = newA;
11    oldB = newB;
12    return result;
13 }
```

2.3.4 本组软件改进

在完成基础功能的基础上，本组针对原始方案中两处已知缺陷进行了软件层面的改进。

(a) 滑动平均滤波——替代固定偏差补偿 原始方案的问题：继电器吸合/断开瞬间，LM35 供电端可能因大电流切换产生电源纹波，导致 ADC 读数突变。最简单的处理方式是在软件中对加热状态下的读数硬编码减去该固定值，但这种方法有明显局限：实际跳变幅度可能发生改变，固定补偿将失效。

本组改进方案：采用 N 点滑动平均滤波器，对连续多次 ADC 采样取平均值后再用于控制和显示。由于滤波器对突变量（脉冲噪声）有平滑作用，继电器切换引起的瞬时扰动被显著抑制，且无需依赖任何经验参数。

滑动平均滤波的输出为：

$$\bar{T}[n] = \frac{1}{N} \sum_{k=0}^{N-1} T[n-k] \quad (10)$$

其中 $T[n]$ 为第 n 次采样的原始温度值， N 为滤波窗口长度。本设计取 $N = 8$ ，在抑制噪声与响应速度之间取得平衡——窗口过小则滤波效果不足，窗口过大则温度显示对实际变化的响应延迟增大。

Listing 4: 本组改进：8 点滑动平均滤波

```

1  #define FILTER_SIZE 8
2
3  float tempBuffer[FILTER_SIZE] = {0}; // 循环缓冲区
4  int   bufIndex = 0;                // 当前写入位置
5
6  // 每次调用读取一次原始 ADC，更新缓冲区并返回平均值
7  float readTempFiltered() {
8      // 读取原始温度（单位：摄氏度）
9      float raw = analogRead(A0) * (5000.0 / 1023.0) / 10.0;
10
11     // 写入循环缓冲区
12     tempBuffer[bufIndex] = raw;
13     bufIndex = (bufIndex + 1) % FILTER_SIZE;
14
15     // 计算窗口内均值
16     float sum = 0;
17     for (int i = 0; i < FILTER_SIZE; i++) sum += tempBuffer[i];
18     return sum / FILTER_SIZE;
19 }
20
21 // 在主循环中替换原 readTemp() 调用
22 void loop() {
23     measureTemp = adjustUnits(readTempFiltered());
24     // ... 其余逻辑不变
25 }
```

(b) 温度趋势指示——增强人机交互 改进动机：原始方案 LCD 第一行仅显示数值 (Actual:25.30oC)，用户无法直观判断当前温度是在上升、下降还是已趋于稳定，必须

等待数值变化才能作出判断，交互体验较差。

改进方案：在温度数值后附加一个趋势字符，实时反映温度的变化方向：

显示字符	含义	触发条件
^	升温中	本次与上次温度差 $> +0.1^{\circ}\text{C}$
v	降温中	本次与上次温度差 $< -0.1^{\circ}\text{C}$
~	温度稳定	温度差在 $\pm 0.1^{\circ}\text{C}$ 以内

阈值 0.1°C 的选取依据：LM35 的固有精度为 $\pm 0.5^{\circ}\text{C}$ ，Arduino ADC（10 位，5V 基准）的温度分辨率约为 $0.49^{\circ}\text{C}/\text{bit}$ 。取 0.1°C 作为趋势判断门限，小于 ADC 分辨率，需配合上面的滑动平均滤波使用，否则噪声会导致趋势符号频繁跳变。

Listing 5: 温度趋势字符计算与显示

```

1 // 全局变量：记录上一次刷新时的温度
2 float lastDisplayTemp = 0;
3
4 // 在 updateDisplay() 函数中加入趋势判断
5 void updateDisplay() {
6     // 计算本次与上次的温度差
7     float delta = measureTemp - lastDisplayTemp;
8     char trend;
9     if (delta > 0.1) trend = '^'; // 升温
10    else if (delta < -0.1) trend = 'v'; // 降温
11    else trend = '~'; // 稳定
12
13    // 更新记录值（仅在刷新显示时更新，与 count==1000 同步）
14    lastDisplayTemp = measureTemp;
15
16    // 覆写第一行：数值 + 趋势符
17    lcd.setCursor(0, 0);
18    lcd.print("Actual:");
19    lcd.print(measureTemp, 2); // 保留两位小数
20    lcd.print(trend);          // 趋势字符占第16列
21    lcd.print("C");
22
23    // 第二行保持原有显示逻辑（Set 温度或 OVERRIDE ON）
24    lcd.setCursor(0, 1);
25    if (override) {
26        lcd.print("OVERRIDE ON ");

```

```

27   } else {
28       lcd.print("Set:");
29       lcd.print(setTemp, 2);
30       lcd.print("oC   ");
31   }
32 }

```

显示效果示例：

Actual:25.30~C
Set:40.00oC

(升温阶段)

Actual:40.12~C
Set:40.00oC

(温度稳定)

滑动平均滤波是趋势显示正常工作的前提。若直接对原始 ADC 值计算差分，噪声会导致 δ 频繁在 $\pm 0.1^{\circ}\text{C}$ 附近抖动，趋势符号将无规律跳变，失去实用价值。滤波后的温度值更加平稳，趋势判断才能真实反映系统的热力学状态。

2.4 原理图与 PCB 设计

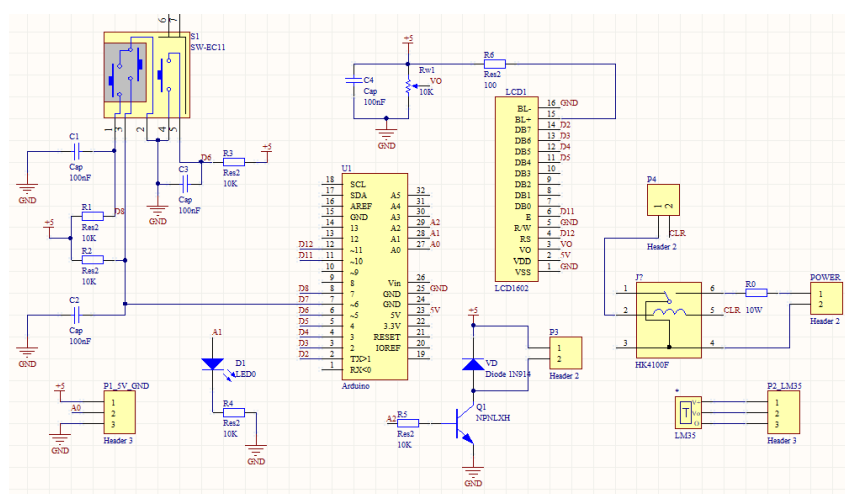


Figure 3: Arduino 温度控制器原理图

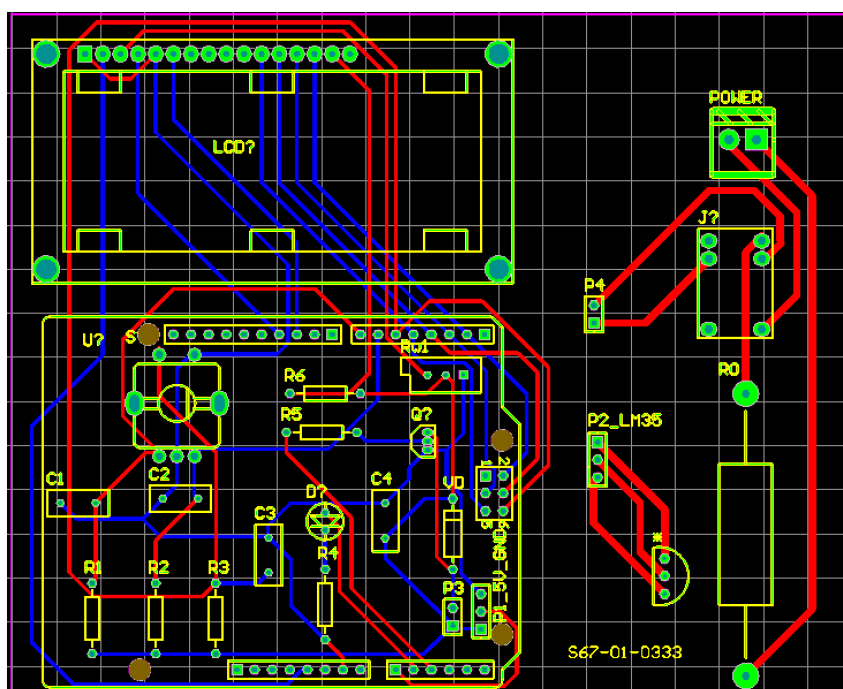


Figure 4: Arduino 温度控制器 pcb

2.4.1 PCB 设计要点

扩展板与主板的电气连接 本设计采用 Arduino UNO 扩展板形式，扩展板通过 P1-P4 排针插合到 Arduino UNO 主板，排针的位置与间距必须与 Arduino 主板的扩展插槽完全对准，否则无法正确插合。在 AD 中布局时，排针连接器是首先确定位置的元件。

布局策略 各功能模块按信号流向排布，形成清晰的功能分区（见表 2）。

Table 2: PCB 功能模块布局

位置	模块	主要元件
左侧	温度设定模块	EC11、 R_1 、 R_2 、 C_1 、 C_2
中央	Arduino 连接模块	P1-P4 排针
右上	温度显示模块	LCD1602
右下	加热与驱动模块	继电器、三极管、 R_T
左下	温度检测模块	LM35

关键布局细节

- LM35 应靠近被测热源（功率电阻）放置，但功率电阻附近不再放置其他元件，避免热干扰；
- 功率电阻应离板安装（留出焊接高度），防止长时间加热导致 PCB 基板碳化变色；

- 流经功率电阻及 12V 电源回路的铜线应加粗 ($\geq 50\text{ mil}$)，降低导线压降；
- 编码器输入端的滤波电容 C_1 、 C_2 尽量紧靠编码器引脚放置，发挥最佳消抖效果。

2.5 本实验学习收获

- (1). 继电器通断引起的 LM35 供电波动（温度示数跳变约 2°C ）通过软件补偿（减去固定偏差量）解决，无需重新设计硬件。说明软件是调整硬件缺陷的有效手段。
- (2). $\pm 0.25^\circ\text{C}$ 的死区设计，有效避免了继电器在设定温度附近的高频切换，延长了器件寿命——这是纯粹的理论学习中很难体会到的工程直觉。
- (3). 排针对准、大电流走线加粗、散热元件离板安装等，都是从这次实践中第一次真正理解其必要性的规范。
- (4). 在模块划分阶段同步考虑硬件连接和软件逻辑，减少了后期返工——例如旋转编码器的引脚分配直接影响中断资源的使用。

总结

两个实验均围绕「设计—仿真/面包板—PCB—调试」的完整流程展开，让我深刻体会到：

- 理论计算是基础，但实际调试中元件容差、寄生参数、电源纹波等因素都会带来偏差，必须保留足够的可调余量（电位器）；
- 遇到调试困难时，回到公式推导往往比反复试错更有效，比如调试 I-V 转换器的过程；
- 良好的 PCB 布局是布线质量的前提，布局阶段节省的时间远多于布线阶段的额外代价；
- 软件与硬件并非截然分开，很多硬件上的缺陷可以通过软件优雅地弥补。

感谢叶险峰老师和施红军老师在实验过程中的悉心指导！